Laboratorio de Git — Interrupciones, `git stash` y resolución de conflictos

Propósito del laboratorio

En este laboratorio vas a trabajar una situación muy cercana a lo que puede ocurrir en el proyecto final del BootCamp: estás desarrollando una tarea, aparece una interrupción, necesitas cambiar de contexto y, al retomar o integrar trabajo, surge un conflicto.

El objetivo es que practiques dos usos distintos de `git stash`:

- un uso **suave**, cuando quieres aparcar trabajo incompleto antes de cambiar de contexto;
- un uso **duro**, cuando Git te exige limpiar el repositorio antes de integrar cambios.

Además, vas a comprobar algo importante: **qué cambios aparca realmente** `stash` y qué ocurre con los archivos ya rastreados frente a los no rastreados.

Modalidad

Este laboratorio puede realizarse de forma **individual** o **por parejas**.

Si trabajáis por parejas, conviene que ambas personas comenten antes de cada paso:

- qué estado tiene el repositorio;
- qué esperáis que ocurra;
- por qué usáis `stash` en ese momento.

Importante

Antes de ejecutar cualquier paso, debes tener claro:

- en qué rama estás;
- si tienes cambios sin confirmar;
- si esos cambios están en archivos ya rastreados o en archivos nuevos;
- si el repositorio está limpio o no.

Objetivos

Al terminar este laboratorio deberías ser capaz de:

- identificar dos contextos distintos donde `git stash` puede resultar útil;
- distinguir entre un uso de comodidad y un uso impuesto por el estado del repositorio;
- comprobar qué ocurre con archivos rastreados y no rastreados al usar `stash`;
- usar `git stash` para aparcar trabajo incompleto;
- recuperar el trabajo aparcado;
- provocar y resolver un conflicto de merge de forma controlada;
- verificar que el repositorio termina en un estado coherente.

Dos contextos reales de uso de `git stash`

Contexto 1. Uso suave

Estás trabajando en una rama, todavía no quieres hacer commit y quieres cambiar de contexto sin dejar trabajo incompleto “suelto”.

Aquí `stash` no siempre es obligatorio, pero puede ser muy útil para:

- dejar el repositorio limpio;
- cambiar de tarea con claridad;
- no hacer commits prematuros.

Contexto 2. Uso duro

Estás trabajando con cambios locales pendientes y quieres integrar o mezclar cambios de otra rama.

En este caso, Git puede impedir la operación o exigirte antes que limpies el estado del repositorio. Aquí `stash` deja de ser una comodidad y pasa a ser una herramienta práctica para desbloquear el flujo.

Contexto	Tipo de uso de `stash`	Idea principal
Cambio de contexto	Suave	Quieres aparcar trabajo incompleto sin hacer commit

Contexto	Tipo de uso de `stash`	Idea principal
Integración con trabajo pendiente	Duro	Git te exige limpiar antes de continuar

Idea central

`git stash` no sustituye a los commits ni a las ramas. Es una herramienta temporal para no perder el control del flujo.

Materiales necesarios

- Git instalado y funcionando;
- terminal;
- editor de texto;
- conocimientos previos de:
  - `git init`
  - `git status`
  - `git add`
  - `git commit`
  - `git branch`
  - `git switch`
  - `git merge`

Estructura del proyecto

Crea una carpeta llamada `laboratorio-stash-conflictos`.

Dentro de ella, crea esta estructura:

```
laboratorio-stash-conflictos/
├─ README.md
├─ guia-flujo.md
├─ incidencias.md
└─ .gitignore
```

Contenido inicial sugerido

Archivo `README.md`

```
# Laboratorio de stash y conflictos

Este repositorio se utiliza para practicar interrupciones de trabajo, uso de
`git stash` y resolución de conflictos de merge.
```

Archivo `guia-flujo.md`

```
# Guía de flujo

## Flujo base

- Crear una rama de trabajo
- Editar archivos
- Hacer commits claros
- Integrar cambios con cuidado
```

Archivo `incidencias.md`

```
# Incidencias

## Incidencia abierta

Hay que mejorar la advertencia principal del flujo de trabajo.
```

Archivo `.gitignore`

```
*.log
temp/
.DS_Store
.vscode/
```

Fase 1. Preparación del repositorio

Tu tarea

1. Inicializa el repositorio.
2. Añade todos los archivos iniciales.
3. Haz un commit inicial con un mensaje coherente.
4. Comprueba que el repositorio está limpio.

Restricción

El commit inicial debe usar Conventional Commits.

Resultado esperado

Debes partir de una base estable sobre la que provocarás interrupciones y conflictos.

> **Comprobación**
>
> Antes de seguir, ejecuta `git status` y asegúrate de que el repositorio está limpio. Ejecuta `git log` para asegurarte de que has hecho un commit y que la rama `main` o `master` está inicializada.

Fase 2. Apertura de una rama de trabajo

Tu tarea

Crea una rama llamada:

`docs/mejora-guia-flujo`

Cámbiate a ella y empieza a trabajar en `guia-flujo.md`.

Modificación requerida

Añade una nueva sección llamada `Buenas prácticas` con, al menos, estas ideas:

- revisar el estado antes de hacer commit;
- trabajar en ramas de tarea;
- revisar antes de integrar.

Situación intencionada

No termines todavía el trabajo. Deja una frase a medio hacer o una sección claramente incompleta.

Resultado esperado

Debes quedarte en una situación donde:

- estás en una rama de trabajo;
- has modificado `guia-flujo.md`;
- todavía no quieres hacer commit;
- el repositorio no está limpio.

Fase 3. Preparar también un archivo no rastreado

Tu tarea

Crea un archivo nuevo llamado `borrador.md` con un contenido breve, por ejemplo:

```
# Borrador

Este archivo se ha creado para comprobar qué aparca realmente `git stash`.
```

Qué debes observar

Ahora deberías tener al menos:

- un archivo **rastreado y modificado**: `guia-flujo.md`;
- un archivo **nuevo no rastreado**: `borrador.md`.

Pregunta de control

Antes de seguir, intenta responder:

> "Si ahora hago `git stash`, ¿desaparecerán los dos cambios o solo algunos?"

Punto clave del laboratorio

Aquí no basta con usar `stash`. Debes comprobar qué cambios aparca realmente.

Fase 4. Primer contexto: uso suave de `stash`

Escenario

Estás trabajando en `docs/mejora-guia-flujo`, todavía no quieres hacer commit y quieres cambiar de contexto para atender una incidencia en `main`.

Aquí vas a usar `stash` como herramienta de comodidad y limpieza, no porque Git te esté bloqueando todavía.

Tu tarea

1. Revisa `git status`.
2. Ejecuta `git stash`.
3. Vuelve a revisar `git status`.
4. Abre el directorio de trabajo y comprueba qué ha ocurrido con:
   - `guia-flujo.md`
   - `borrador.md`

Qué debes comprobar

Debes fijarte en:

- si desaparecen los cambios de `guia-flujo.md`;
- si desaparece también `borrador.md`;
- si el repositorio queda limpio;
- si `stash` ha aparcado solo los cambios rastreados o todos.

Preguntas de observación

Debes poder responder:

1. ¿Qué ha aparcado realmente `stash` en esta primera ejecución?
2. ¿Qué ha pasado con el archivo no rastreado?
3. ¿El repositorio ha quedado completamente limpio o no?

> ⚠ **No pases por alto esta comprobación**
>
> Una parte importante del laboratorio es precisamente observar que `stash` no trata siempre igual los archivos rastreados y los no rastreados.

Fase 5. Completar el aparcamiento incluyendo archivos no rastreados

Escenario

Ahora quieres aparcar también el archivo nuevo que se ha quedado fuera.

Tu tarea

1. Vuelve a revisar `git status`.
2. Usa una variante de `stash` que incluya también archivos no rastreados.
3. Vuelve a revisar `git status`.
4. Comprueba qué ha ocurrido con `borrador.md`.

Qué debes observar

Debes comprobar:

- si ahora sí desaparece también el archivo no rastreado;
- si el repositorio queda completamente limpio;
- qué diferencia hay entre esta situación y la anterior.

Resultado esperado

Debes haber comprobado por experiencia que:

- una forma básica de `stash` aparca los cambios rastreados;
- para incluir ciertos archivos nuevos hace falta una variante más completa del comando.

> ✓ **Idea clave**
>
> No todo lo que ves en el directorio de trabajo se aparca del mismo modo con `stash`. Debes entender qué tipo de cambio tienes delante.

Fase 6. Cambio de contexto y trabajo urgente en `main`

Tu tarea

1. Cámbiate a `main`.
2. Modifica `incidencias.md` añadiendo debajo de `Incidencia abierta` este bloque:

```
> [!warning]
> Antes de integrar cambios, asegúrate de no tener trabajo local sin
```

```
confirmar.
```

3. Revisa el estado.
4. Haz un commit coherente.
5. Comprueba el historial.

Resultado esperado

Debes haber podido cambiar de contexto sin hacer commit del trabajo incompleto de la otra rama.

Fase 7. Recuperar el trabajo aparcado

Tu tarea

1. Vuelve a la rama `docs/mejora-guia-flujo`.
2. Recupera el trabajo que habías aparcado.
3. Revisa el estado.
4. Comprueba el contenido de:
   - `guia-flujo.md`
   - `borrador.md`

Qué debes observar

Debes comprobar:

- si reaparecen los cambios del archivo rastreado;
- si reaparece el archivo no rastreado;
- si el repositorio vuelve a quedar con trabajo pendiente;
- si ese trabajo sigue sin formar parte del historial.

> **Idea importante**
>
> Recuperar con `stash` no significa que el trabajo quede confirmado. Solo vuelve al directorio de trabajo.

Fase 8. Segundo contexto: uso duro de `stash`

Escenario

Ahora estás de nuevo en `docs/mejora-guia-flujo`, has recuperado tu trabajo y quieres integrar cambios de `main` en esta rama. Pero todavía tienes trabajo local pendiente.

Este es el contexto fuerte del laboratorio: Git no debe mezclar sin más una integración con un estado local inestable.

Tu tarea

1. Asegúrate de que `guia-flujo.md` sigue teniendo cambios sin commit.
2. Añade además una pequeña modificación nueva en `guia-flujo.md`.
3. Intenta integrar `main` en tu rama.
4. Observa el mensaje de Git.
5. Usa `git stash` para aparcar el trabajo pendiente.
6. Reintenta la integración.

Qué debes observar

Debes entender que aquí `stash` no es simplemente comodidad:

- Git necesita que limpies el estado antes de integrar;
- estás usando `stash` para desbloquear una operación del flujo;
- integrar con trabajo local desordenado es una mala práctica.

> **Idea clave**
>
> Este es el uso duro de `stash`: no lo usas solo porque te venga bien, sino porque el estado del repositorio te obliga a ordenar el trabajo antes de continuar.

Fase 9. Preparación del conflicto

Escenario

Ahora vas a provocar un conflicto real entre `main` y `docs/mejora-guia-flujo`.

Paso A. Cambio en `main`

Estando en `main`, modifica en `guia-flujo.md` la línea:

```
- Integrar cambios con cuidado
```

para que pase a ser:

```
- Integrar cambios con cuidado y revisar siempre la rama actual antes del
merge
```

Haz un commit con mensaje coherente.

Paso B. Cambio en la rama de trabajo

Vuelve a `docs/mejora-guia-flujo` y, en esa misma línea original, deja esta versión:

```
- Integrar cambios con cuidado y evitar mezclar varias tareas en la misma rama
```

Haz ahora el commit que te faltaba en la rama de trabajo.

> ⚠ **Punto clave**
>
> El conflicto depende de que ambas ramas modifiquen la misma línea de forma incompatible.

Fase 10. Provocar el conflicto de merge

Tu tarea

1. Estando en `docs/mejora-guia-flujo`, intenta integrar `main`.
2. Observa qué ocurre.
3. Ejecuta `git status`.
4. Abre el archivo afectado.

Resultado esperado

Debe aparecer un conflicto real en `guia-flujo.md`.

Qué deberías encontrar

Deberías encontrar una estructura similar a esta:

```
<<<<<<< HEAD
- Integrar cambios con cuidado y evitar mezclar varias tareas en la misma rama
=======
- Integrar cambios con cuidado y revisar siempre la rama actual antes del
merge
>>>>>>> main
```

Significado de los marcadores

Marcador	Significado
`<<<<<<< HEAD`	Comienza la versión de la rama actual
`=======`	Separa ambas versiones
`>>>>>>> main`	Termina la versión procedente de la otra rama

Fase 11. Resolver el conflicto

Tu tarea

Debes decidir qué contenido final dejar en `guia-flujo.md`.

Recomendación

En este laboratorio se recomienda crear una versión combinada, por ejemplo:

```
- Integrar cambios con cuidado, revisar siempre la rama actual antes del merge
y evitar mezclar varias tareas en la misma rama
```

Pasos

1. elimina los marcadores del conflicto;
2. deja una única versión final coherente;
3. guarda el archivo;
4. revisa `git status`;
5. añade el archivo resuelto;
6. completa la integración.

Qué debes comprobar

- que el archivo ya no contiene marcadores;
- que la línea final tiene sentido;
- que el repositorio deja de estar en situación de conflicto.

> **Criterio técnico**
>
> Resolver un conflicto no consiste en borrar marcas. Consiste en dejar el contenido correcto.

Fase 12. Verificación final

Tu tarea

Al terminar el laboratorio, revisa:

- el historial;
- el contenido final de `guia-flujo.md`;
- el estado del repositorio.

Tabla de comprobación final

Comprobación	Sí / No
He usado `stash` como herramienta de cambio de contexto	
He comprobado qué aparca `stash` en archivos rastreados	
He comprobado qué ocurre con un archivo no rastreado	
He usado una variante de `stash` para incluir también archivos no rastreados	
He entendido por qué Git me pedía limpiar el repositorio antes de integrar	
He usado `stash` como herramienta fuerte para desbloquear una integración	
He provocado un conflicto real de merge	
He identificado correctamente los marcadores de conflicto	
He resuelto el conflicto con un resultado coherente	
El repositorio termina en un estado limpio	

Entregable esperado

Debes conservar como evidencia:

- el repositorio `laboratorio-stash-conflictos`;
- el historial resultante;
- el archivo `reflexion-stash-conflictos.md`.

Preguntas de reflexión

Crea un archivo llamado `reflexion-stash-conflictos.md` y responde brevemente:

1. ¿Qué diferencia has visto entre el uso suave y el uso duro de `git stash`?
2. ¿Qué cambios ha aparcado `stash` en su forma básica?
3. ¿Qué ha ocurrido con el archivo no rastreado y por qué?
4. ¿Por qué Git puede impedirte integrar cambios si tienes trabajo local sin confirmar?
5. ¿Qué diferencia hay entre usar `stash` y hacer un commit?
6. ¿Por qué ha aparecido el conflicto en este laboratorio?
7. ¿Qué significado tienen `<<<<<<<`, `=======` y `>>>>>>>`?
8. ¿Qué criterio has seguido para decidir la versión final del conflicto?
9. ¿Qué te parece más importante para el proyecto final del BootCamp:
   - saber usar `stash`,
   - saber resolver conflictos,
   - o entender bien el estado del repositorio antes de actuar?

> **Cierre reflexivo**
>
> Este laboratorio no busca que memorices dos comandos aislados. Busca que entiendas una parte muy real del trabajo con Git: interrumpir una tarea sin perder el control, limpiar el estado antes de integrar y resolver colisiones de cambios con criterio.

Recursos de apoyo

- Git Book en español: `https://git-scm.com/book/es/v2`
- Documentación oficial de Git: `https://git-scm.com/docs/git/es`

- Learn Git Branching: `https://learngitbranching.js.org/?locale=es_ES`
- Explain Git with D3: `https://onlywei.github.io/explain-git-with-d3/`
- Git Exercises: `https://gitexercises.fracz.com/`